

# Agile Methodology

## Story Points and Estimation

# Learning Objectives

By the end of this discussion, you will be able to:

- Understand what story points are and why Agile teams use them
- Explain why story points are not time-based estimates
- Identify key factors that contribute to story points
- Describe how relative estimation works
- Apply common story point scales (Fibonacci, modified Fibonacci)
- Understand velocity and its role in planning

- Scheduling an increment plan depends on the following input:
  - Requirements
    - What is it you need to develop?
  - Work estimates
    - How much effort is involved?
  - Priorities
    - What's most important to the customer?

# Estimating Techniques

- Agile estimating is similar to traditional estimating.
- Choice to use one or many of existing estimating techniques:
  - ideal days
  - story points
  - function points ...
- Ideal days measure estimates in days it would take to complete a requirement if there were no overhead.
  - One person's ideal days may not be the same as another person's ideal days.
  - Ideal days are easy to understand but may set the wrong expectations.
  - You may not want to communicate estimates in days with your client.
- Story points are a measure of relative size - key topic of discussion.
  - Fibonacci sequence: 1, 2, 3, 5, 8, 13

# What Are Story Points?

- Story points are a way Agile teams estimate how **big or complex** a piece of work is
- They are a unit of **relative effort**, not a measure of time
- Teams avoid saying:
  - “This will take 6 hours”
- Instead, teams say:
  - “This feels about twice as hard as that other story”

# Why Not Estimate in Hours or Days?

Time-based estimates often fail because:

- Different people work at different speeds
- Interruptions and context switching distort time estimates
- Unknowns are hard to express in hours
- Pressure to “commit” to a time creates false certainty

Story points deliberately **separate estimation from scheduling**.

# The Core Idea of Story Points

A story point represents the **combined weight** of three factors:

- 1 How much work is involved
- 2 How hard the work is
- 3 How uncertain or risky the work is

Story points capture **effort, complexity, and uncertainty together**.

# Factor 1: Amount of Work

This includes:

- Number of tasks involved
- Size of code changes
- Amount of testing required
- Documentation or review effort

More work generally means more story points — but not always by itself.



## Factor 2: Complexity

Complexity refers to:

- Difficult algorithms or logic
- Intricate integrations
- Hard-to-understand existing code
- Tight coupling with other components

Complex work increases story points even if the task is small.

## Factor 3: Uncertainty

Uncertainty comes from:

- Vague or evolving requirements
- New technologies or tools
- Lack of prior experience
- External dependencies

More uncertainty means higher estimation risk — and higher story points.

# Story Points Are Relative

- Story points only make sense **relative to other stories**
- Teams usually select a small story as a reference
- All other stories are compared against that baseline

Example:

- If Story A is 2 points
- And Story B feels twice as hard
- Then Story B is estimated as 4 points

# Why Relative Estimation Works Better

Relative estimation:

- Is cognitively easier than absolute estimation
- Reduces arguments over exact durations
- Encourages team discussion and shared understanding
- Improves consistency over time

# Common Story Point Scales

Most teams use non-linear scales such as:

- Fibonacci sequence: 1, 2, 3, 5, 8, 13, 21
- Modified Fibonacci: 1, 2, 3, 5, 8, 13, 20, 40
- T-shirt sizes: XS, S, M, L, XL (mapped later)

The gaps grow larger to reflect increasing uncertainty.

# Why Fibonacci Numbers?

Fibonacci-style scales:

- Acknowledge that uncertainty grows with size
- Prevent false precision for large stories
- Force teams to think in rough categories

If you cannot decide between 8 and 13, the story is probably too large.

# Example Story Point Estimation

Assume:

- “Add a simple validation rule” = 1 point

Then:

- “Add a form with validation” = 3 points
- “Add form + backend integration” = 5 points
- “New workflow with external API” = 8 or 13 points

# What Is Velocity?

Velocity is:

- The number of story points a team completes in a sprint -  
Alternatively, Velocity is a measure of how much work is performed in an increment.
- Velocity can be expressed in ideal days or story points.
- Measured empirically, not predicted
- An initial velocity is set at the start of the project to schedule work to increments.
- This can be total work estimate divided by the number of increments.
- Unique to each team

Example:

- Sprint 1: 18 points
- Sprint 2: 20 points
- Sprint 3: 22 points



# Using Velocity for Planning

Velocity is measured at the end of each increment to track progress and forecast time of completion.

Velocity helps teams:

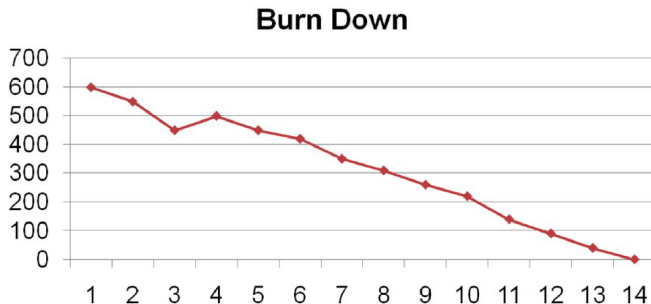
- Forecast how much work fits in a sprint
- Plan releases realistically
- Avoid over commitment

Velocity should stabilize over time — but never be forced - means **Velocity forecasts become more accurate as iterations are completed.**

- Story points can be roughly translated to ideal days by dividing the work estimate by the number of story points.
- Less work will be done in the first couple of increments

## Measuring Progress

- Important to define “done” upfront
- Always aim at completing the increment's target velocity



# What Story Points Are NOT

Story points are NOT:

- Hours or days
- A performance metric for individuals
- Comparable across teams
- A contractual commitment

# Common Misuses of Story Points

- Converting story points to hours
- Comparing velocities between teams
- Using points to evaluate individuals
- Inflating estimates to appear “safe”

These behaviors defeat the purpose of story points.

# Key Takeaways

- Story points measure relative effort, not time
- Story points are best produced by the development team - team feels more engaged.
- They combine work, complexity, and uncertainty
- They trigger team discussions around design approach and understanding of requirements.
- Relative estimation improves consistency
- Velocity emerges from actual team performance
- Proper use builds trust and predictability
- Given the relative nature of story points, they may be more appropriate for communicating size to the client.
- Using multiple estimating techniques can sometimes lead to better results as they force you to think about discrepancies between estimates.